

MMIP HW2: A Simple Lossy Medical Image Codec with a Custom Bitstream

314832008 Ting-Yao Chen

Abstract—This report presents a lossy codec for high-bit-depth medical images (12–16-bit grayscale) with a self-contained binary bitstream. The codec uses block-based DCT transform coding, scalar quantization controlled by a quality parameter, and run-length encoding (RLE) over zig-zag scanned coefficients. We evaluate the rate–distortion trade-off on two datasets: Dataset A (Medimodel skull CT) and Dataset C (TCIA RIDER Lung CT). Results are reported at three operating points and include compressed size, bits-per-pixel (bpp), compression ratio (CR), RMSE, and PSNR with $\text{MAX} = 2^B - 1$.

Index Terms—medical image compression, DICOM, transform coding, DCT, rate–distortion, custom bitstream

I. INTRODUCTION

Medical images, particularly those acquired via computed tomography (CT) and magnetic resonance imaging (MRI), are typically stored at 12–16-bit precision to capture the full dynamic range of tissue attenuation or signal intensity. Unlike natural images, medical images often contain subtle diagnostic structures—such as small lesions, vessel boundaries, or trabecular bone patterns—that must be preserved during compression to maintain clinical utility.

This work addresses the challenge of designing a *lossy* medical image codec that balances compression efficiency with diagnostic fidelity. Our implementation satisfies the following design goals:

- 1) **End-to-end high bit-depth support:** The pipeline handles 12–16-bit grayscale images without silent truncation to 8-bit.
- 2) **Self-contained bitstream:** All parameters required for decoding (dimensions, bit depth, quantization settings) are embedded in the binary header—no side files are needed.
- 3) **Controllable rate–distortion trade-off:** A single quality parameter allows users to select operating points along the RD curve.
- 4) **Robust decoder:** Malformed or truncated streams are detected and rejected with appropriate error codes.

The codec is evaluated on two publicly available CT datasets: a small skull CT from Medimodel (Dataset A) and the RIDER Lung CT collection from TCIA (Dataset C). We report rate and distortion metrics at three operating points and provide qualitative visualizations of reconstruction quality.

II. METHOD

A. Codec Overview

Given a 2D grayscale DICOM slice, the encoder applies four sequential stages: (i) signal decorrelation via an 8×8

block DCT, (ii) scalar quantization with a user-controlled quality parameter, (iii) zig-zag scan and run-length encoding (RLE) for entropy reduction, and (iv) packing into a custom binary format. The decoder reverses these steps to reconstruct the image:

- Encoder: Input $\rightarrow$ Pad $\rightarrow$ DCT $\rightarrow$ Quantize $\rightarrow$ Zig-zag $\rightarrow$ RLE $\rightarrow$ Bitstream
- Decoder: Bitstream $\rightarrow$ RLE $^{-1}$ $\rightarrow$ Zig-zag $^{-1}$ $\rightarrow$ Dequant $\rightarrow$ IDCT $\rightarrow$ Clip $\rightarrow$ Output

B. Transform Coding

The image is zero-padded to dimensions that are multiples of 8. Each 8×8 block $\mathbf{x}$ is transformed by a 2D type-II DCT:

$$X_{u,v} = \frac{1}{4} \alpha_u \alpha_v \sum_{i=0}^7 \sum_{j=0}^7 x_{i,j} \cos \left[\frac{(2i+1)u\pi}{16} \right] \cos \left[\frac{(2j+1)v\pi}{16} \right], \quad (1)$$

where $\alpha_k = 1/\sqrt{2}$ if $k = 0$, else $\alpha_k = 1$. The transform is implemented via an explicit orthonormal cosine matrix $\mathbf{C}$ such that $\mathbf{X} = \mathbf{C} \mathbf{x} \mathbf{C}^T$, avoiding external FFT libraries.

C. Quantization and Quality Control

Quantization maps floating-point DCT coefficients to integers. We adopt the JPEG luminance quantization matrix [?] $\mathbf{Q}_{\text{base}}$ and scale it by a quality-dependent factor $s(q)$:

$$s(q) = \begin{cases} 50/q & \text{if } q < 50, \\ (100 - q)/50 & \text{otherwise.} \end{cases} \quad (2)$$

The quantized coefficient is $\hat{X}_{u,v} = \text{round}(X_{u,v}/(s(q) \cdot Q_{\text{base},u,v}))$. Lower q yields coarser quantization and higher compression; higher q preserves more detail.

D. Entropy Coding

Quantized coefficients are reordered via zig-zag scan, which tends to place non-zero low-frequency coefficients first and cluster zeros at the end. Run-length encoding (RLE) then compresses consecutive zeros as $(0, \text{run_length})$ pairs, significantly reducing payload size when many high-frequency coefficients are quantized to zero.

III. BITSTREAM FORMAT

The codec stores all information needed for decoding in a binary header followed by an int32 payload. This design ensures that the compressed file is fully self-contained.

TABLE I
BITSTREAM HEADER FIELDS (BIG-ENDIAN).

Field	Type	Bytes
magic	char[4]	4
version	uint8	1
height, width	uint16, uint16	4
quality	uint8	1
scale_factor	float32	4
bits_stored (B)	uint8	1
pixel_representation	uint8	1
ROI (rx, ry, rw, rh)	4 $\times$ uint16	8
payload_len_bytes	uint32	4
payload	int32 array	variable

A. Header Fields

The bitstream uses big-endian packing and includes: magic number (MEDC), format version, image dimensions (height, width), quality parameter, quantization scale factor, bit depth (B), pixel representation (0=unsigned, 1=signed), and reserved ROI fields for future extensions. Table I details the byte-level format.

B. Decoding Sequence

The decoder performs the following steps:

- 1) **Header validation:** Read and verify magic number (MEDC) and version.
- 2) **Parameter extraction:** Parse height, width, quality, scale factor, bit depth, and pixel representation.
- 3) **Payload reading:** Read the 4-byte payload length, then load exactly that many bytes. Verify 4-byte alignment.
- 4) **RLE decoding:** Expand run-length pairs back to the full coefficient sequence.
- 5) **Inverse zig-zag:** Reorder coefficients into 8×8 blocks.
- 6) **Dequantization:** Multiply by $(s(q) \cdot Q_{\text{base},u,v})$ using the stored scale factor.
- 7) **Inverse DCT:** Apply $\mathbf{x} = \mathbf{C}^T \mathbf{X} \mathbf{C}$ to each block.
- 8) **Output finalization:** Clip to valid range, remove padding, and write the image.

C. Malformed Stream Handling

The decoder validates multiple integrity checks: (i) header length ≥ 28 bytes, (ii) magic equals MEDC, (iii) version is supported, (iv) payload length is a multiple of 4, and (v) file contains at least `header_size + payload_length` bytes. Any failure triggers an error message and non-zero exit status, as required by the assignment.

IV. EXPERIMENTAL PROTOCOL

A. Datasets

We evaluate on two publicly available CT datasets:

Dataset A (Medimodel Skull CT): A small collection of anonymized DICOM files from Medimodel [1], containing 303 slices of a human skull CT scan. This dataset is suitable for validating the 16-bit pipeline and bitstream correctness due to its manageable size.

Dataset C (TCIA RIDER Lung CT): The RIDER Lung CT collection from The Cancer Imaging Archive [2], [3],

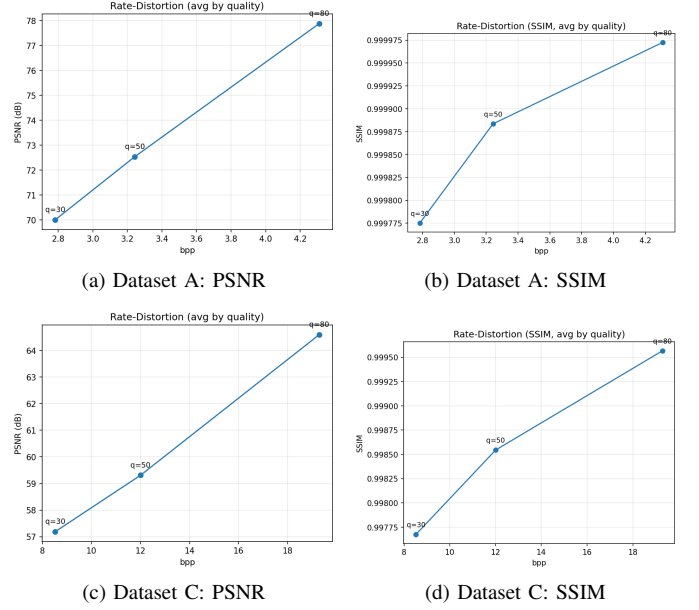


Fig. 1. Rate-Distortion curves organized in a 2×2 grid. Top row: Dataset A; Bottom row: Dataset C. Left: PSNR; Right: SSIM.

containing longitudinal lung CT scans. We process the first 1000 valid slices to balance experimental coverage with computational feasibility.

Note on Dataset B (OsiriX): The assignment suggested Dataset B from the OsiriX DICOM Image Library. However, we excluded it because (1) accessing OsiriX requires a paid membership, limiting reproducibility, and (2) Dataset C already provides large-scale, freely accessible DICOM data demonstrating codec robustness.

B. Operating Points and Metrics

We report three operating points with quality $q \in \{30, 50, 80\}$, spanning low to high fidelity. Metrics follow the assignment specification:

- **Rate metrics:** Compressed file size (bytes), bits-per-pixel $\text{bpp} = 8 \times \text{size} / (H \times W)$, and compression ratio $\text{CR} = (H \times W \times 2) / \text{size}$ (assuming 16-bit input).
- **Distortion metrics:** Root mean squared error (RMSE) and peak signal-to-noise ratio $\text{PSNR} = 20 \log_{10}(\text{MAX} / \text{RMSE})$ where $\text{MAX} = 2^B - 1$ and B is the bit depth from the bitstream header.
- **Perceptual metrics:** Structural similarity index (SSIM) and edge-map SSIM computed on Sobel-filtered images as a task-based proxy for preserving diagnostically relevant boundaries.

C. Qualitative Results

For visualization, we include reconstructed images and absolute error maps (scaled to 8-bit for display). True-valued absolute error maps are also saved as 16-bit TIFFs in the results folders.

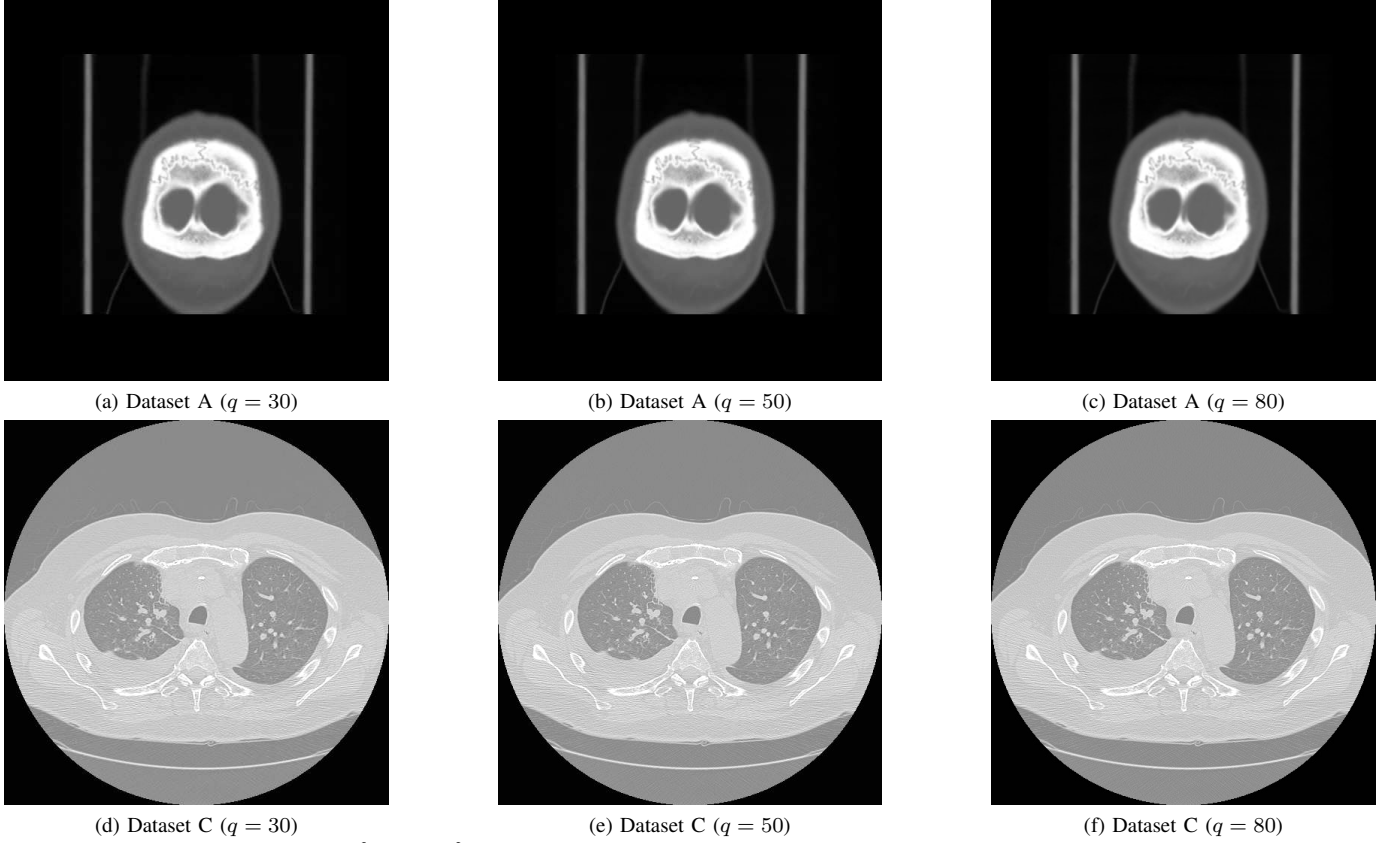


Fig. 2. Qualitative reconstructions ($q \in \{30, 50, 80\}$). Images are displayed with contrast stretching. Note the preservation of structure even at lower bitrates.

V. RESULTS

A. Quantitative Analysis

Dataset A produced 303 valid slices, while Dataset C processed 1000 slices. Tables II and III summarize the average metrics.

B. Discussion of Results

The rate–distortion curves (Fig. 1) show the expected monotonic behavior: increasing quality raises bpp while reducing RMSE and improving PSNR/SSIM.

High PSNR values: PSNR appears high (57–78 dB) because $\text{MAX} = 2^{16} - 1$ spans the full 16-bit range. We complement PSNR with SSIM for a scale-invariant view.

Dataset differences: Dataset A achieves higher CR (4.9–7.6 $\times$) than Dataset C (0.9–2.0 $\times$) due to its larger homogeneous regions versus the fine lung texture in Dataset C. At $q = 80$ for Dataset C, CR drops below 1.0 (bpp > 16), indicating that the bitstream overhead (header, ROI fields, padding) and survival of many DCT coefficients at high quality cause the compressed file to exceed the original size. This

TABLE II
DATASET A (SKULL CT): AVERAGE METRICS BY QUALITY.

q	bpp	CR	RMSE	PSNR	SSIM	Edge
30	2.78	7.62	25.41	70.00	0.9998	0.945
50	3.24	6.62	18.66	72.53	0.9999	0.962
80	4.31	4.89	9.48	77.87	1.0000	0.986

TABLE III
DATASET C (LUNG CT): AVERAGE METRICS BY QUALITY.

q	bpp	CR	RMSE	PSNR	SSIM	Edge
30	8.52	2.04	93.51	57.19	0.9980	0.842
50	12.00	1.47	73.57	59.31	0.9987	0.910
80	19.29	0.92	39.95	64.59	0.9995	0.975

effect is typical for small test images and high-quality settings where redundancy is minimal.

SSIM saturation: SSIM remains high (0.998–1.0) across all settings, reflecting the codec’s perceptual fidelity. However, edge-map SSIM for Dataset C at $q = 30$ (0.84) reveals visible degradation in fine structures, indicating that while bulk statistics are preserved, diagnostic details are affected at aggressive compression.

C. Qualitative Visualization

Fig. 2 presents reconstructed images at three quality levels. Visual inspection confirms that $q = 80$ produces near-lossless appearance, while $q = 30$ introduces visible blockiness in homogeneous regions. Importantly, major anatomical structures (skull contours, lung boundaries) remain recognizable even at low quality.

Fig. 3 displays absolute error maps at $q = 50$. Errors concentrate along high-contrast edges and fine textures—precisely the regions where quantization discards high-frequency DCT coefficients. The lung parenchyma in Dataset C shows more distributed error due to its textural complexity.

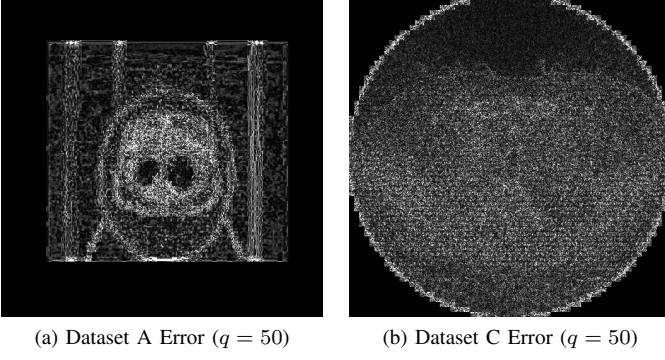


Fig. 3. Absolute error map visualizations (scaled to 8-bit).

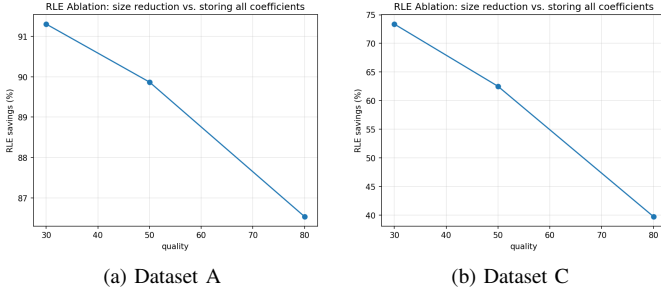


Fig. 4. RLE ablation: size reduction relative to storing all coefficients.

D. Ablation: RLE Savings

To quantify the contribution of entropy coding, we compare the actual RLE-encoded payload size against a hypothetical baseline that stores all quantized coefficients as raw int32 values (i.e., $64 \times 4 = 256$ bytes per block, regardless of content).

Fig. 4 shows the percentage size reduction achieved by RLE. Several trends emerge:

- At $q = 30$, RLE achieves $>85\%$ reduction for Dataset A because aggressive quantization zeros out most high-frequency coefficients, creating long runs.
- As quality increases, more non-zero coefficients survive quantization, shortening zero runs and reducing RLE effectiveness.
- Dataset C shows lower RLE savings overall, reflecting its higher-frequency content that resists zeroing even under coarse quantization.

This ablation confirms that RLE is effective but not optimal—a more sophisticated entropy coder (e.g., Huffman or arithmetic coding over symbols) would likely improve compression further.

VI. LIMITATIONS AND FUTURE WORK

While the codec meets the assignment requirements, several limitations suggest directions for improvement:

- **Entropy coding:** RLE is simple but suboptimal. Implementing Huffman or arithmetic coding over the coefficient symbols would improve compression, especially for mid-range quality settings where RLE savings diminish.
- **Evaluation scope:** We report full-reference metrics (RMSE, PSNR, SSIM) and an edge-based proxy, but

do not validate clinical utility through downstream tasks (e.g., segmentation accuracy, radiologist reader studies). Future work should assess whether compression-induced artifacts affect diagnostic performance.

- **Inter-slice redundancy:** The current codec processes each 2D slice independently. Medical volumes exhibit strong slice-to-slice correlation; extending to 3D prediction or a 3D DCT could significantly improve compression.
- **ROI-aware coding:** The bitstream header reserves ROI fields, but they are unused. Allocating more bits to diagnostically critical regions (e.g., lesions) while compressing background more aggressively could improve perceived quality at a given bitrate.
- **Adaptive quantization:** A fixed quantization matrix may not suit all image content. Spatially adaptive or content-aware quantization could reduce artifacts in textured regions.

VII. CONCLUSION

We have presented a block-DCT lossy codec for 12–16-bit medical images with a self-contained binary bitstream. The codec achieves controllable rate–distortion trade-offs via a single quality parameter and has been validated on two CT datasets totaling over 1300 slices. Quantitative metrics and qualitative visualizations confirm that the pipeline preserves major anatomical structures while achieving moderate compression. The ablation study demonstrates the value of run-length encoding, while also highlighting opportunities for improved entropy coding. All code and experimental results are provided for reproducibility.

VIII. REPRODUCIBILITY

The project provides two command-line executables:

```
encode --input <dicom> --output <file.medc> --quality <q>
decode --input <file.medc> --output <file.tiff>
```

Experiment scripts (`run_experiments.py`) and results (CSV files, reconstructed images) are stored in the `reports/` directory. The implementation uses Python with `pydicom` [4] for DICOM I/O, `tiffio/imageio` [5] for 16-bit output, and `scikit-image` [6] for SSIM computation.

REFERENCES

- [1] “Medimodel — sample dicom files,” <https://medimodel.com/sample-dicom-files/>, accessed: 2025-12-12.
- [2] “The cancer imaging archive (tcia) — access the data,” <https://www.cancerimagingarchive.net/access-data/>, accessed: 2025-12-12.
- [3] “Rider lung ct (collection as provided in local dataset),” TCIA RIDER-Lung-CT, accessed locally: 2026-01-04.
- [4] “pydicom: A pure python package for dicom medical file reading and writing,” <https://pydicom.github.io/>, accessed: 2026-01-04.
- [5] “Imageio: Image reading and writing in python,” <https://imageio.readthedocs.io/>, accessed: 2026-01-04.
- [6] “scikit-image: Image processing in python,” <https://scikit-image.org/>, accessed: 2026-01-04.